



U.A. Tecnologías para el Desarrollo de Aplicaciones Web PHP – Hypertext Preprocessor



PHP es un popular lenguaje de secuencias de comandos de propósito general que es especialmente adecuado para el desarrollo web. Lo usamos para hacer que el servidor genere una salida dinámica, salida que es diferente cada vez que un navegador solicita una página.

Estructura de PHP

Para activar los comandos de PHP, necesitamos colocar la etiqueta:

```
<?php
```

Secciones de código.....

```
?>
```

➤ Utilización de comentarios

Hay dos maneras de añadir comentarios al código PHP:

//Este es un comentario, al inicio para ocultar una línea de código en el proceso de depuración hasta que se requiera,

\$x += 10; //Incrementa la variable x en 10 al final de una línea de código para describir lo que hace o,

/ Este es un
comentario de varias
líneas */*

Para comentar varias líneas.

Sintaxis básica

En PHP no debemos olvidar que todos los comandos se cierran con un “;” **p.e.:** *\$x += 10;*

Debemos colocar el signo \$ delante de todas las variables (para que el analizador de PHP actúe más rápido, ya sabe al instante cuando se encuentra con una variable, independientemente del tipo de dato).

Algunas asignaciones de variables:

```
<?php
$contador = 1;
$cadena = "Hola"; // Cadena de caracteres (string)
$miArreglo = array('Uno', 'Dos', 'Tres', 'Cuatro', 'Cinco', 'Seis');

?>
```



U.A. Tecnologías para el Desarrollo de Aplicaciones Web PHP – Hypertext Preprocessor



Para mostrar el contenido de una variable en pantalla utilizaremos lo siguiente:

```
echo $cadena;
```

Recuerda que, para ver los resultados inmediatos, debes guardar tus archivos con extensión .php en tu carpeta de publicación y acceder a ellos desde el navegador web como:

```
http://localhost/miArchivo.php
```

En caso de que hayas cambiado el puerto asignado al servidor a cualquier otro que no sea el 80, no olvides hacer referencia a él después de la palabra localhost, de la siguiente forma:

```
http://localhost:8080/miArchivo.php
```

Reglas para la creación de variables:

1. Los nombres de las variables, después del símbolo \$ deben comenzar con una letra o el carácter _.
2. Los nombres de las variables solo pueden contener los caracteres a-z, A-Z, 0-9 y _.
3. Los nombres de las variables no pueden contener espacios. Si una variable tiene que incluir más de una palabra, podemos separarlas con _ o bien utilizar la notación del camello (p.e. \$nombreUsuario).
4. Los nombres de las variables distinguen entre mayúsculas y minúsculas.

Operadores

Los operadores permiten especificar operaciones matemáticas como sumas, restas, multiplicaciones y divisiones. Pero también existen otros tipos de operadores, como la cadena de caracteres, la comparación y los operadores lógicos.

- Aritméticos: realizan cálculos matemáticos (+, -, *, /, %, ++, -- y **).
- De asignación: asignan valores a variables (=, +=, -=, *=, /=, .=, %=).
- De comparación: se utilizan generalmente dentro de un constructor, por ejemplo una declaración if en la que se deben de comparar dos elementos (==, !=, >, <, >=, <=, <>, ===, !==).
- Lógicos: se utilizan generalmente para combinar los resultados de dos de los operadores de comparación, recibe dos entradas verdaderas o falsas y produce un resultado verdadero o falso (&&, ||, !, xor).

La concatenación de cadenas en PHP es con un . Por ejemplo:

```
echo "Tienes ". $numMensajes ." mensajes sin leer.";
```

De igual forma, puedes añadir una cadena a otra mediante .=, así:

```
echo $noticia .= $noticiaImp;
```



U.A. Tecnologías para el Desarrollo de Aplicaciones Web

PHP – Hypertext Preprocessor



Tipos de cadenas

PHP admite dos tipos de cadenas que se denotan por el tipo de comillas que usemos. Si deseamos asignar una cadena literal y preservar el contenido exacto, debemos utilizar comillas simples, así:

```
$info = 'Preface variable with a $like this: $variable';
```

o si deseamos incluir el valor de una variable dentro de una cadena, se hace mediante cadenas con comillas dobles:

```
echo "Tienes $numMensajes mensajes sin leer."; //Sustitución de variables
```

Tipificación de variables

PHP es un lenguaje débilmente tipado. Significa que las variables no se tienen que declarar antes de utilizarlas, y que PHP siempre convierte variables al tipo requerido por su contexto cuando se accede a ellas.

```
//Conversión automática de un número a una cadena  
$number = 12345 * 67890;  
echo substr($number, 3, 1);
```

```
//Conversión automática de una cadena a un número  
$pi = "3.1415927";  
$radio = 5;  
echo $pi * ($radio * $radio);
```

Constantes

Las constantes son similares a las variables, contienen información a la que se accede después, excepto que son lo que parecen: constantes. Una vez que hayas definido una, su valor se fija para el resto del programa y no se puede cambiar.

Un ejemplo de una constante es la ubicación de la carpeta de publicación del servidor. Definirías una constante así:

```
define("ROOT_LOCATION", "usr/local/www/");
```

Para leer el contenido de la variable, solo tienes que referirte a ella como una variable normal (pero las constantes no están precedidas por el símbolo \$):

```
$directorio = ROOT_LOCATION;
```



U.A. Tecnologías para el Desarrollo de Aplicaciones Web PHP – Hypertext Preprocessor



Constantes predefinidas

PHP tiene docenas de constantes predefinidas, algunas conocidas como constantes mágicas, las cuales identificarás con dos guiones bajos al inicio y al final para que no intentes nombrar una de tus constantes con alguno que ya está ocupado, entre ellas tenemos:

`__LINE__`, `__FILE__`, `__DIR__`, `__FUNCTION__`, `__CLASS__`, `__METHOD__`,
`__NAMESPACE__`.

Estas variables encuentran su uso práctico en la depuración, cuando se necesita insertar una línea de código para ver si el flujo del programa llega hasta ella:

```
echo "Esta es la línea " . __LINE__ . " del archivo " . __FILE__;
```

Funciones

Las funciones contienen las secciones de código que realizan una tarea en concreto. Crear una función con un código que se utiliza a menudo no solo acorta el programa y lo hace más legible, sino que también añade funcionalidad extra, ya que se pueden pasar parámetros a las funciones para que operen de manera diferente. También pueden devolver valores al código desde el que se les llama.

Para crear una función hay que declararla de la siguiente manera:

```
function fechaLarga($timestamp){  
    return date("l F jS Y", $timestamp);  
}  
echo fechaLarga(time());
```

Variables locales: son variables que se crean dentro de una función y a las que solo se puede acceder mediante esa función. Generalmente son variables temporales que se utilizan para almacenar resultados parcialmente procesados antes de que la función proporcione un resultado.

Variables globales: son variables a las cuales podemos acceder desde cualquier parte de código. Para acceder a ellas hay que añadir la palabra clave *global*: `global $is_logged_in;`

Variables estáticas: son variables locales definidas dentro de una función, pero de las cuales quieres mantener su valor para la próxima vez que se llame a la función, por esa razón las declaras como estáticas. Solo se pueden inicializar con valores predeterminados no con el resultado de una expresión:



U.A. Tecnologías para el Desarrollo de Aplicaciones Web PHP – Hypertext Preprocessor



```
function test(){  
    static $count = 0;  
    echo $count;  
    $count++;  
}
```

Variables superglobales: significa que las proporciona el entorno PHP (predefinidas), pero son globales dentro del programa, accesibles desde todas partes. Contienen mucha información útil sobre el programa en ejecución y su entorno. Están estructuradas como arreglo asociativo:

\$GLOBALS, \$_SERVER, \$_GET, \$_POST, \$_FILES, \$_COOKIE, \$_SESSION, \$_REQUEST y \$_ENV.

Accedemos a la página de referencia de la siguiente forma:

```
$came_from = $_SERVER['HTTP_REFERER'];
```